PYTHON ACTIVITY-BASED LEARNING

Complete Documentation for 10 Randomly Selected Case Studies

Included for each case: Problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, and VS Code-style dark-theme screenshots.

Selected case study numbers

42, 15, 68, 29, 57, 9, 81, 24, 48, 94

Prepared from the case-study descriptions in the supplied Python Activity-Based Learning PDF. Where the source did not specify rates or rules, sample assumptions are stated explicitly.

CASE STUDY 42 – Student Marks Matrix Analysis

1. Problem Statement

Analyse a matrix of student marks across subjects, calculate student-wise totals and averages, calculate subject-wise averages, and identify the top-performing student.

2. Objectives

• Store student marks in a nested list.
• Calculate each student's total and average.
• Calculate the average mark for each subject.
• Identify the student with the highest total.

3. Assumptions

• All marks are numeric and out of 100.
• Each student has a mark for every listed subject.
• The sample data contains at least one student and one subject.

4. Algorithm

1. Start and define student names, subject names and the marks matrix.
2. For each student, calculate the row total and average.
3. For each subject, add marks from all students and calculate its average.
4. Find the student with the greatest total.
5. Display student-wise and subject-wise results, then stop.

5. Flowchart



6. Pseudocode

```
START\nSET student names, subject names and marks matrix\nFOR each student:\n    total = sum of that student's marks\n    average =
```

7. Python Source Code

```
students = ["Akhil", "Bharath", "Charan", "Divya"]
subjects = ["Python", "Maths", "English"]

marks = [
    [85, 90, 78],
    [76, 82, 88],
    [92, 95, 89],
    [68, 74, 80]
]

print("STUDENT MARKS ANALYSIS")

for i in range(len(students)):
```

```
    total = sum(marks[i])
    average = total / len(subjects)
    print("\\nStudent:", students[i])
    print("Marks:", marks[i])
    print("Total:", total)
    print("Average:", round(average, 2))

print("\\nSUBJECT-WISE AVERAGES")
for j in range(len(subjects)):
    total = 0
    for i in range(len(students)):
        total += marks[i][j]
    print(subjects[j], "Average:", round(total / len(students), 2))

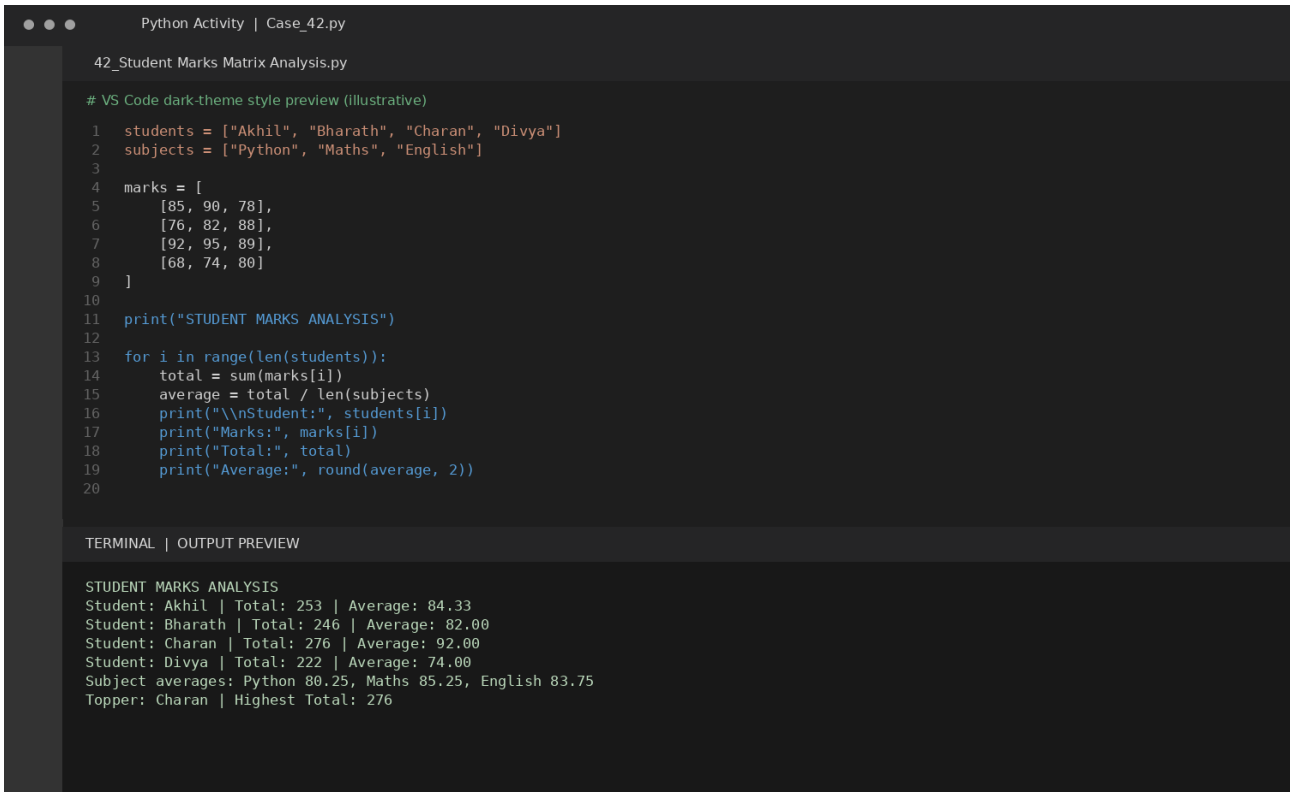
highest = -1
topper = ""
for i in range(len(students)):
    total = sum(marks[i])
    if total > highest:
        highest = total
        topper = students[i]

print("\\nTopper:", topper)
print("Highest Total:", highest)
```

8. Test Cases

Test	Input / Condition	Expected Result
Akhil marks	85, 90, 78	Total 253; average 84.33
Divya marks	68, 74, 80	Total 222; average 74.00
Topper	All four students	Charan; total 276

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
STUDENT MARKS ANALYSIS
Student: Akhil | Total: 253 | Average: 84.33
Student: Bharath | Total: 246 | Average: 82.00
Student: Charan | Total: 276 | Average: 92.00
Student: Divya | Total: 222 | Average: 74.00
Subject averages: Python 80.25, Maths 85.25, English 83.75
Topper: Charan | Highest Total: 276
```

10. Conclusion

The program for Case Study 42 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.

CASE STUDY 15 – Smart Parking Fee Management

1. Problem Statement

Calculate parking charges from entry and exit times, vehicle categories, overnight parking and a long-duration surcharge while validating time inputs.

2. Objectives

• Accept vehicle type and entry/exit times.
• Validate hour and minute values.
• Calculate elapsed parking time, including crossing midnight.
• Calculate the parking fee using the vehicle rate and surcharge rule.

3. Assumptions

• Rates are assumed as car Rs.30/hour, bike Rs.15/hour and bus Rs.50/hour.
• Any started hour is charged as a full hour.
• A surcharge of Rs.100 applies when parking exceeds 12 hours.
• The parking duration is at most 24 hours; equal entry and exit times are treated as 24 hours.

4. Algorithm

1. Start and define hourly rates for vehicle categories.
2. Read vehicle type and entry/exit hour and minute.
3. Validate vehicle and time ranges.
4. Convert times to minutes; if exit is not later than entry, add 24 hours.
5. Round duration up to whole hours and calculate fee.
6. Add surcharge if duration exceeds 12 hours; display bill.

5. Flowchart



6. Pseudocode

```
START\nSET vehicle rates\nINPUT vehicle, entry time and exit time\nIF vehicle or time is invalid: DISPLAY error\nELSE:\n    CONVERT
```

7. Python Source Code

```
print("SMART PARKING FEE MANAGEMENT")

rates = {"car": 30, "bike": 15, "bus": 50}
vehicle = input("Vehicle (car/bike/bus): ").lower()

if vehicle not in rates:
```

```
    print("Invalid vehicle type")
else:
    eh = int(input("Entry hour (0-23): "))
    em = int(input("Entry minute (0-59): "))
    xh = int(input("Exit hour (0-23): "))
    xm = int(input("Exit minute (0-59): "))

    valid = (0 <= eh <= 23 and 0 <= xh <= 23
             and 0 <= em <= 59 and 0 <= xm <= 59)

    if not valid:
        print("Invalid time")
    else:
        entry = eh * 60 + em
        exit_time = xh * 60 + xm

        if exit_time <= entry:
            exit_time += 24 * 60

        duration = exit_time - entry
        hours = (duration + 59) // 60
        amount = hours * rates[vehicle]

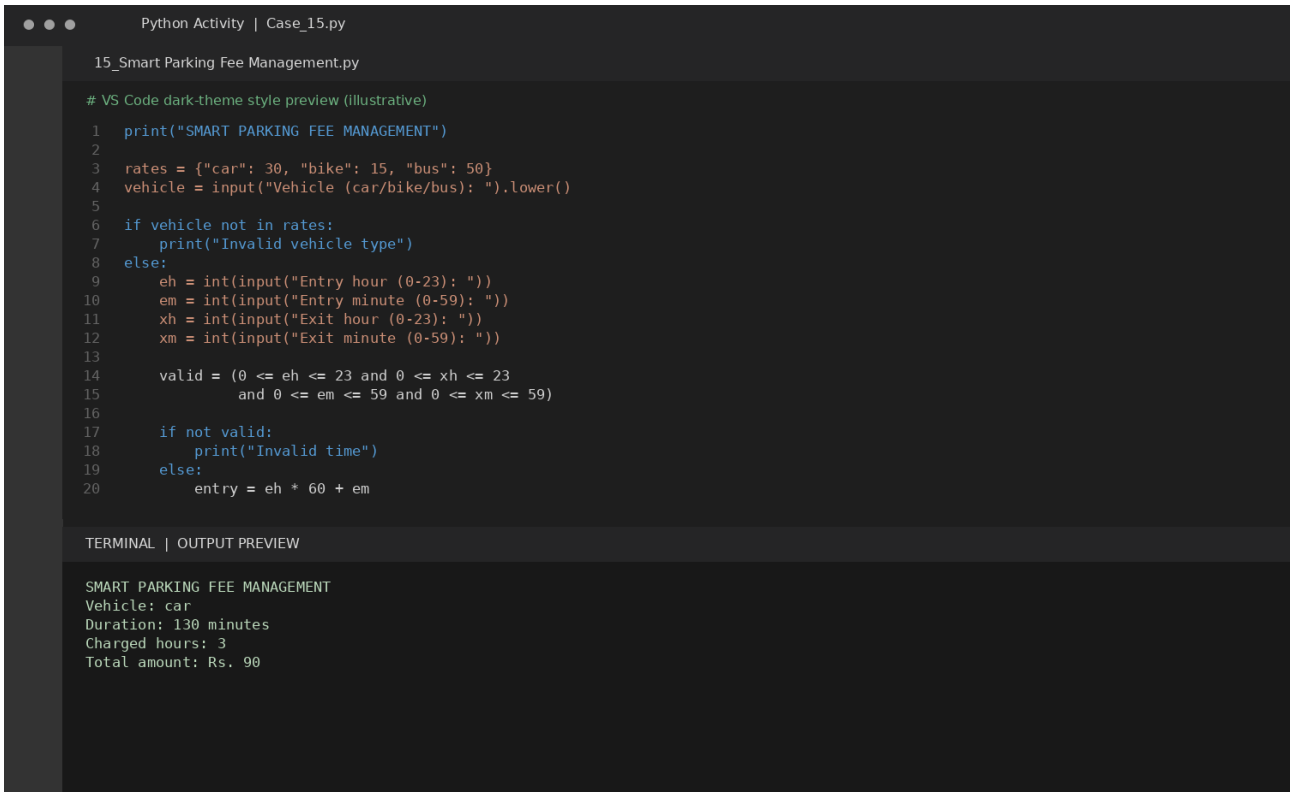
        if hours > 12:
            amount += 100

        print("Vehicle:", vehicle)
        print("Duration:", duration, "minutes")
        print("Charged hours:", hours)
        print("Total amount: Rs.", amount)
```

8. Test Cases

Test	Input / Condition	Expected Result
Car, 2 h 10 min	car; 09:00 to 11:10	3 hours × 30 = Rs.90
Bike across midnight	bike; 23:30 to 00:30	1 hour × 15 = Rs.15
Invalid time	car; 25:00 entry	Invalid time

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
SMART PARKING FEE MANAGEMENT
Vehicle: car
Duration: 130 minutes
Charged hours: 3
Total amount: Rs. 90
```

10. Conclusion

The program for Case Study 15 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.

CASE STUDY 68 – Energy Conservation Before-and-After Analysis

1. Problem Statement

Compare appliance-wise electricity consumption before and after an energy-saving measure, calculate savings and savings percentages, and rank appliances by savings.

2. Objectives

• Store appliance readings in lists.
• Calculate unit savings for each appliance.
• Calculate percentage savings.
• Calculate overall consumption reduction.

3. Assumptions

• Readings are in kWh for the same measurement period.
• Before values are positive.
• The sample after-values do not exceed before-values.

4. Algorithm

1. Start and define appliance names and before/after readings.
2. For each appliance, subtract after-reading from before-reading.
3. Calculate percentage savings using the before-reading.
4. Accumulate total before and after consumption.
5. Calculate overall savings and display results.

5. Flowchart



6. Pseudocode

```
START\nSET appliance names, before list and after list\nSET total_before = 0, total_after = 0\nFOR each appliance:\n    saving = bef
```

7. Python Source Code

```
appliances = ["Fan", "Light", "AC", "TV"]
before = [120, 80, 300, 100]
after = [90, 50, 220, 85]

total_before = 0
total_after = 0
print("ENERGY CONSERVATION ANALYSIS")
```

```
for i in range(len(appliances)):
    saving = before[i] - after[i]
    percentage = saving / before[i] * 100
    total_before += before[i]
    total_after += after[i]

    print(appliances[i])
    print("Before:", before[i], "kWh")
    print("After:", after[i], "kWh")
    print("Saving:", saving, "kWh")
    print("Saving percentage:", round(percentage, 2), "%")

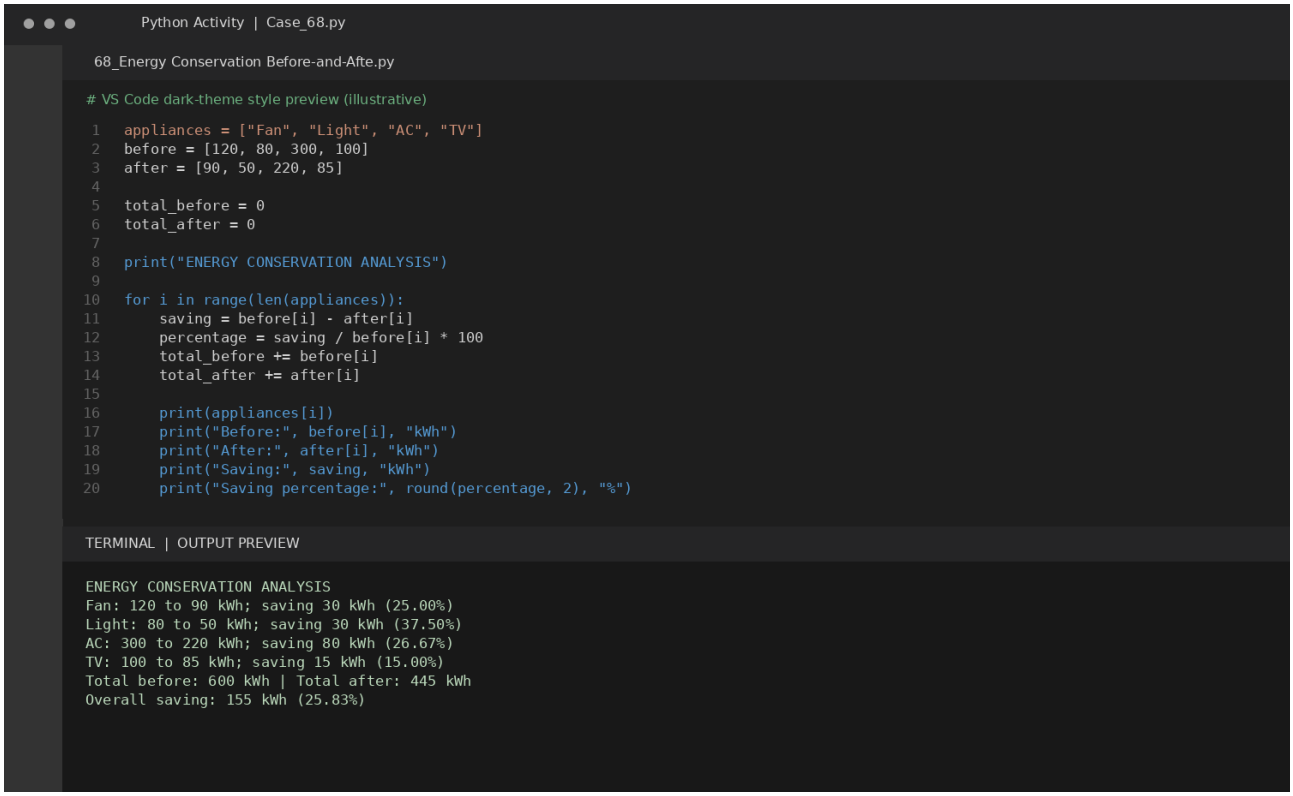
total_saving = total_before - total_after
overall = total_saving / total_before * 100

print("Total before:", total_before, "kWh")
print("Total after:", total_after, "kWh")
print("Total saving:", total_saving, "kWh")
print("Overall saving:", round(overall, 2), "%")
```

8. Test Cases

Test	Input / Condition	Expected Result
Fan	120 before; 90 after	30 kWh saved; 25.00%
AC	300 before; 220 after	80 kWh saved; 26.67%
Overall	600 before; 445 after	155 kWh saved; 25.83%

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
ENERGY CONSERVATION ANALYSIS
Fan: 120 to 90 kWh; saving 30 kWh (25.00%)
Light: 80 to 50 kWh; saving 30 kWh (37.50%)
AC: 300 to 220 kWh; saving 80 kWh (26.67%)
TV: 100 to 85 kWh; saving 15 kWh (15.00%)
Total before: 600 kWh | Total after: 445 kWh
Overall saving: 155 kWh (25.83%)
```

10. Conclusion

The program for Case Study 68 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.

CASE STUDY 29 – Password Policy Validation Engine

1. Problem Statement

Validate sample password strings against minimum length, uppercase, lowercase, digit, special-character and repeated-character rules, and report each failed condition.

2. Objectives

• Accept a password string.
• Check its length and character categories.
• Detect repeated characters.
• Display all validation failures or confirm that the password meets the sample policy.

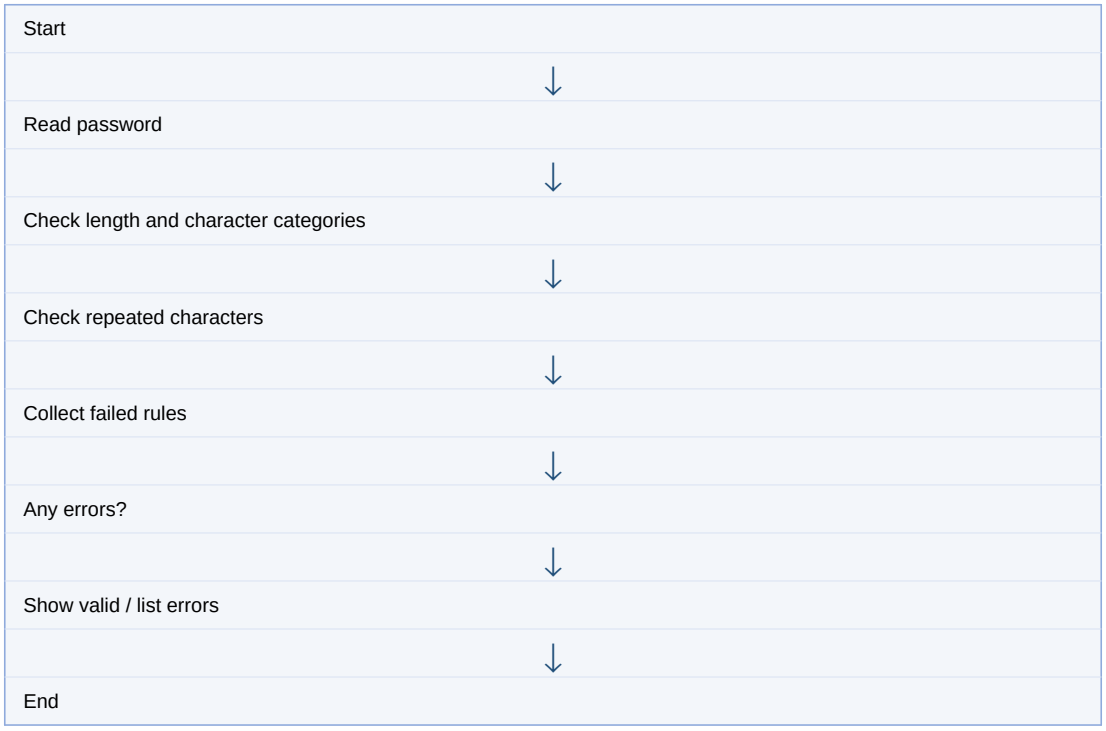
3. Assumptions

• Minimum length is 8 characters.
• At least one uppercase, lowercase, digit and non-alphanumeric character is required.
• Every character must be unique for this sample policy.
• This is an educational validator, not a secure password-storage system.

4. Algorithm

1. Start and read a password.
2. Create flags for uppercase, lowercase, digit and special character.
3. Check each character and update the flags.
4. Add an error for each missing requirement.
5. Check length and repeated characters.
6. Display valid status or all errors.

5. Flowchart



6. Pseudocode

```
START\nINPUT password\nSET uppercase, lowercase, digit, special flags to FALSE\nFOR each character in password:\n    UPDATE matching
```

7. Python Source Code

```
print("PASSWORD POLICY VALIDATION")
password = input("Enter password: ")

errors = []

if len(password) < 8:
```

```
    errors.append("Minimum 8 characters required")

upper = lower = digit = special = False

for ch in password:
    if ch.isupper():
        upper = True
    elif ch.islower():
        lower = True
    elif ch.isdigit():
        digit = True
    else:
        special = True

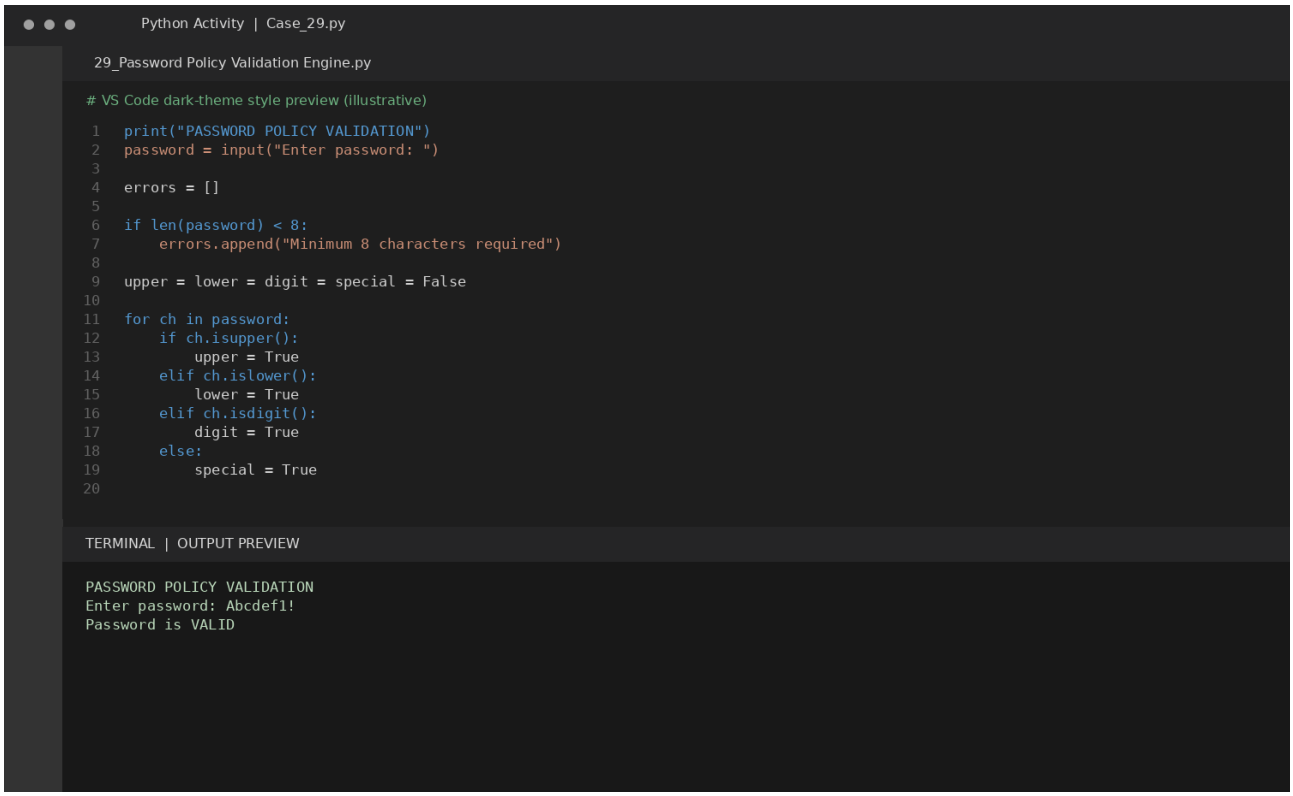
if not upper:
    errors.append("At least one uppercase letter required")
if not lower:
    errors.append("At least one lowercase letter required")
if not digit:
    errors.append("At least one digit required")
if not special:
    errors.append("At least one special character required")
if len(password) != len(set(password)):
    errors.append("Repeated characters are not allowed")

if len(errors) == 0:
    print("Password is VALID")
else:
    print("Password is INVALID")
    for error in errors:
        print("-", error)
```

8. Test Cases

Test	Input / Condition	Expected Result
Valid sample	Abcdef1!	Valid
Missing uppercase	abcdef1!	Invalid: uppercase missing
Repeated character	Aabcdef1!	Invalid: repeated character

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
PASSWORD POLICY VALIDATION
Enter password: Abcdef1!
Password is VALID
```

10. Conclusion

The program for Case Study 29 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.

CASE STUDY 57 – Smart Canteen Order and Stock Management

1. Problem Statement

Process successive canteen orders, calculate bills, update available inventory, reject unavailable quantities, and summarize daily sales and remaining stock.

2. Objectives

• Display a menu with prices and stock.
• Accept repeated customer orders.
• Validate item selections and quantities.
• Update stock and total sales after successful orders.
• Print the closing sales and stock report.

3. Assumptions

• The sample menu and prices are illustrative.
• Stock is measured in item units.
• No tax or discount is applied.
• Entering 0 ends order entry.

4. Algorithm

1. Start and initialize item names, prices, stock and sales total.
2. Display the menu in a loop.
3. Read item choice; stop if choice is 0.
4. Validate choice and quantity.
5. If enough stock exists, calculate bill, reduce stock and add to sales.
6. Otherwise reject the order.
7. Display final sales and remaining stock.

5. Flowchart



6. Pseudocode

```
START\nINITIALIZE items, prices, stock, total_sales\nREPEAT\n    DISPLAY menu and stock\n    INPUT choice\n    IF choice = 0: EXIT L
```

7. Python Source Code

```
items = ["Idli", "Dosa", "Meals", "Juice"]
prices = [30, 50, 80, 25]
stock = [50, 40, 30, 60]
total_sales = 0

while True:
    print("\\nCANTEEN MENU")
    for i in range(len(items)):
        print(i + 1, items[i], "Rs.", prices[i],
              "Stock:", stock[i])

    choice = int(input("Item number (0 to exit): "))
    if choice == 0:
        break
    if choice < 1 or choice > len(items):
        print("Invalid item selection")
        continue

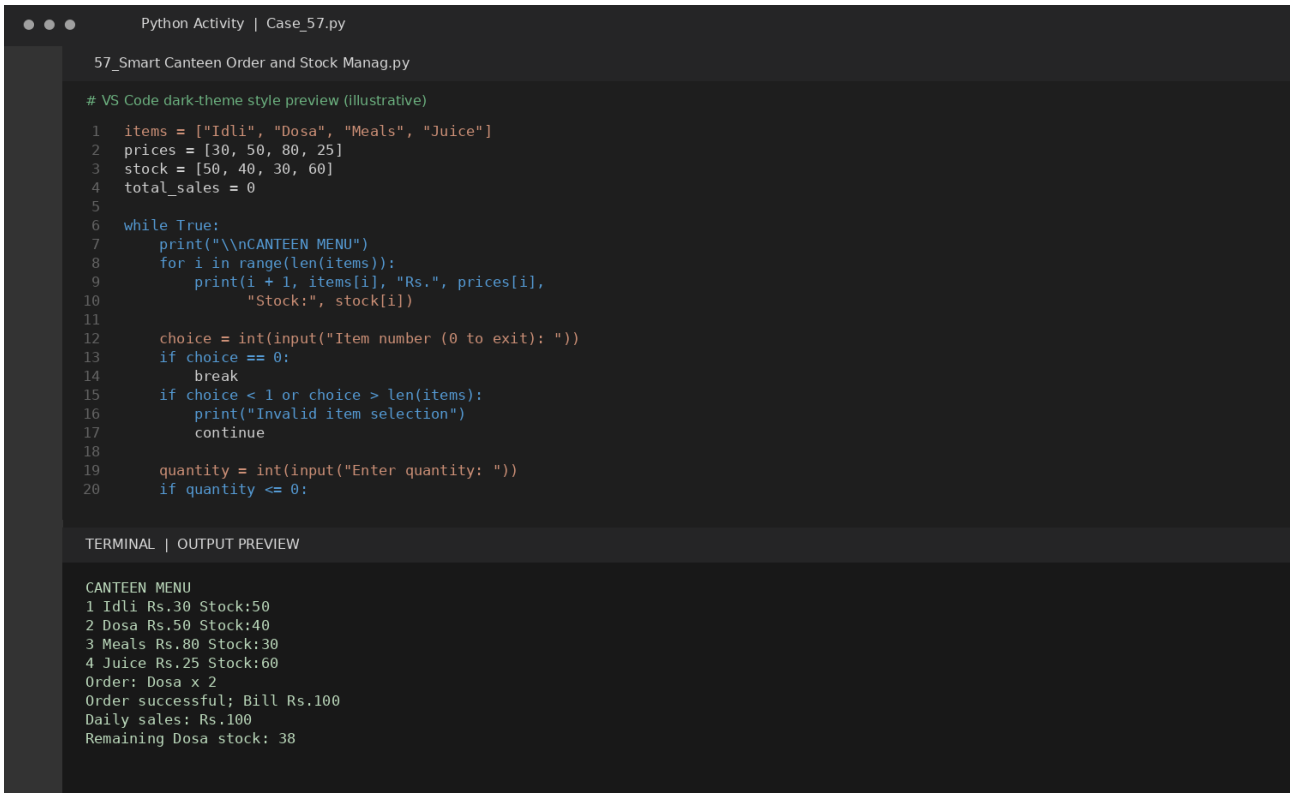
    quantity = int(input("Enter quantity: "))
    if quantity <= 0:
        print("Invalid quantity")
    elif quantity > stock[choice - 1]:
        print("Insufficient stock")
    else:
        index = choice - 1
        bill = prices[index] * quantity
        stock[index] -= quantity
        total_sales += bill
        print("Order successful:", items[index])
        print("Quantity:", quantity)
        print("Bill: Rs.", bill)

print("Daily sales: Rs.", total_sales)
print("Remaining stock:")
for i in range(len(items)):
    print(items[i], ":", stock[i])
```

8. Test Cases

Test	Input / Condition	Expected Result
Normal order	Dosa, quantity 2	Bill Rs.100; Dosa stock becomes 38
Insufficient stock	Meals, quantity 35	Order rejected; stock remains 30
Exit	Choice 0	Closing report displayed

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
CANTEEN MENU
1 Idli Rs.30 Stock:50
2 Dosa Rs.50 Stock:40
3 Meals Rs.80 Stock:30
4 Juice Rs.25 Stock:60
Order: Dosa x 2
Order successful; Bill Rs.100
Daily sales: Rs.100
Remaining Dosa stock: 38
```

10. Conclusion

The program for Case Study 57 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.

CASE STUDY 9 – Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert travel expenses in different currencies using supplied exchange rates, add conversion fees, compare total spending with a budget, and identify the largest expense category.

2. Objectives

• Accept a travel budget in INR.
• Convert USD, EUR and GBP expenses using sample rates.
• Calculate a conversion fee for each expense.
• Compare total spending with the budget.
• Identify the largest converted expense.

3. Assumptions

• Sample exchange rates are USD=Rs.88, EUR=Rs.102 and GBP=Rs.117 per unit; replace with rates supplied by the instructor.
• A 2% conversion fee is applied to each converted amount.
• The program considers one expense amount per currency.
• Rates are illustrative and not live exchange rates.

4. Algorithm

1. Start and define currencies, rates and conversion fee.
2. Read the INR travel budget.
3. For each currency, read the expense amount.
4. Convert amount to INR and calculate 2% fee.
5. Store final converted expense and update total.
6. Compare total against budget and find the largest expense.
7. Display the report.

5. Flowchart



6. Pseudocode

```
START\nSET currencies and sample exchange rates\nINPUT budget in INR\nFOR each currency:\n    INPUT expense amount\n    converted =
```

7. Python Source Code

```
currencies = ["USD", "EUR", "GBP"]
rates = [88, 102, 117]  # illustrative INR rates
expenses = []
total_expense = 0

budget = float(input("Enter travel budget in INR: "))

for i in range(len(currencies)):
    print("Currency:", currencies[i])
    amount = float(input("Enter expense amount: "))

    converted = amount * rates[i]
    fee = converted * 0.02
    final_amount = converted + fee

    expenses.append(final_amount)
    total_expense += final_amount

    print("Converted: Rs.", round(converted, 2))
    print("Fee: Rs.", round(fee, 2))
    print("Final: Rs.", round(final_amount, 2))

print("Total expense: Rs.", round(total_expense, 2))

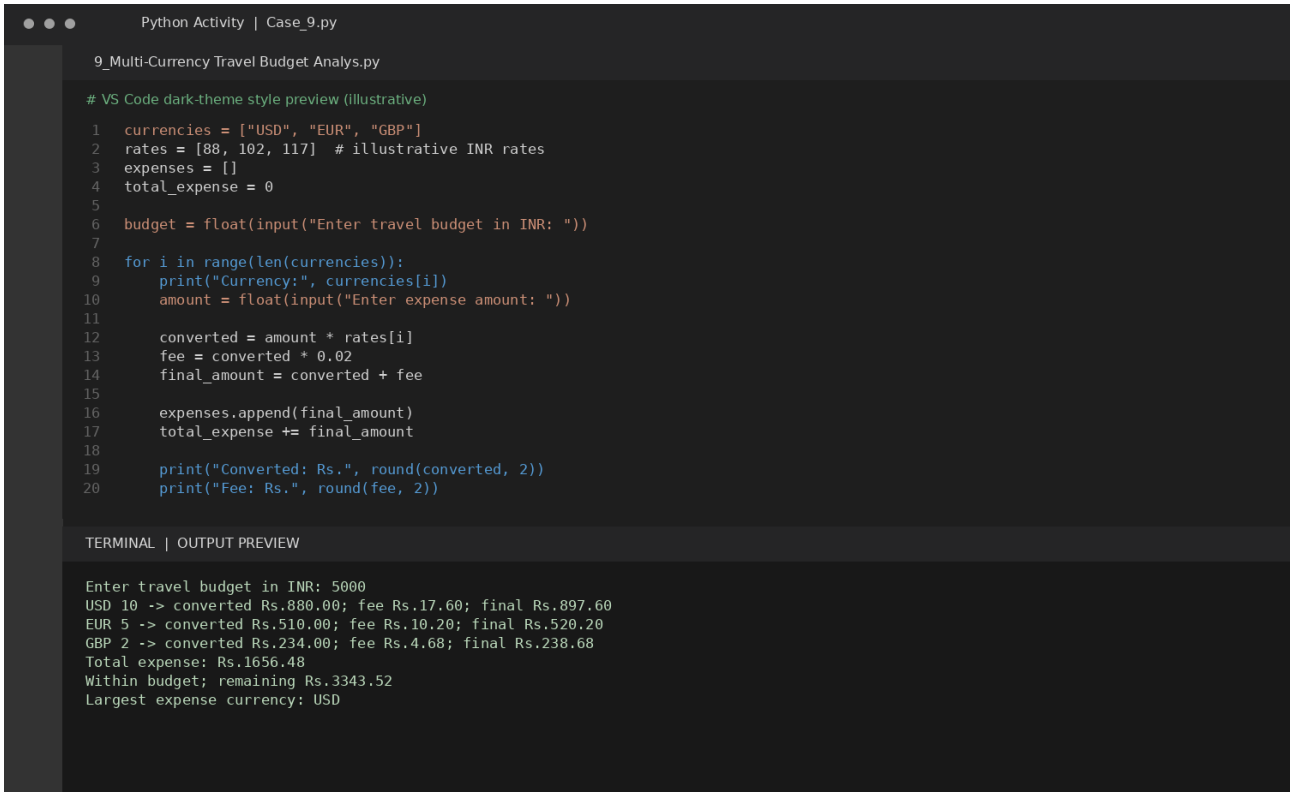
if total_expense <= budget:
    print("Within budget")
    print("Remaining: Rs.", round(budget - total_expense, 2))
else:
    print("Over budget")
    print("Excess: Rs.", round(total_expense - budget, 2))

largest = expenses.index(max(expenses))
print("Largest expense currency:", currencies[largest])
```

8. Test Cases

Test	Input / Condition	Expected Result
USD expense	10 USD at Rs.88	Rs.897.60 including 2% fee
Within budget	Budget Rs.5000; total Rs.3000	Within budget; Rs.2000 remains
Over budget	Budget Rs.1000; total Rs.2000	Over budget by Rs.1000

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
Enter travel budget in INR: 5000
USD 10 -> converted Rs.880.00; fee Rs.17.60; final Rs.897.60
EUR 5 -> converted Rs.510.00; fee Rs.10.20; final Rs.520.20
GBP 2 -> converted Rs.234.00; fee Rs.4.68; final Rs.238.68
Total expense: Rs.1656.48
Within budget; remaining Rs.3343.52
Largest expense currency: USD
```

10. Conclusion

The program for Case Study 9 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.

CASE STUDY 81 – Campus Event Seating and Entry Validation

1. Problem Statement

Generate seating assignments using a nested list, preserve reserved seats, validate tickets against registration IDs, detect duplicate entries, and report vacant seating capacity.

2. Objectives

• Represent a seating plan as a nested list.
• Preserve reserved seats.
• Allocate registered student IDs to available seats.
• Validate event entry and detect duplicate ticket use.
• Calculate vacant seats.

3. Assumptions

• The sample hall has 3 rows and 4 seats per row.
• The text RESERVED marks seats unavailable for allocation.
• The registered IDs are sample records.
• A ticket can be accepted only once.

4. Algorithm

1. Start and create the seating matrix and registered ID list.
2. For each registered student, scan rows and seats for the first empty seat.
3. Allocate the student or report no seat.
4. Display the seating matrix.
5. Read entry ticket IDs and validate registration and duplicate status.
6. Count vacant seats and display the summary.

5. Flowchart



6. Pseudocode

```
START\nSET seating matrix with empty and RESERVED positions\nSET registered student IDs and entered IDs = empty list\nFOR each regis
```

7. Python Source Code

```
seats = [
    [None, None, "RESERVED", None],
    [None, None, None, None],
    [None, "RESERVED", None, None]
]
```

```
registered = ["S101", "S102", "S103", "S104",
              "S105", "S106", "S107", "S108"]
entered = []
allocated = 0

for student in registered:
    found = False
    for i in range(len(seats)):
        for j in range(len(seats[i])):
            if seats[i][j] is None:
                seats[i][j] = student
                print(student, "-> Row", i + 1, "Seat", j + 1)
                allocated += 1
                found = True
                break
        if found:
            break
    if not found:
        print(student, "No seat available")

print("SEATING PLAN")
for row in seats:
    print(row)

number = int(input("Number of students entering: "))
for i in range(number):
    ticket = input("Ticket ID: ")
    if ticket not in registered:
        print("Invalid ticket")
    elif ticket in entered:
        print("Duplicate entry")
    else:
        entered.append(ticket)
        print("Entry successful")

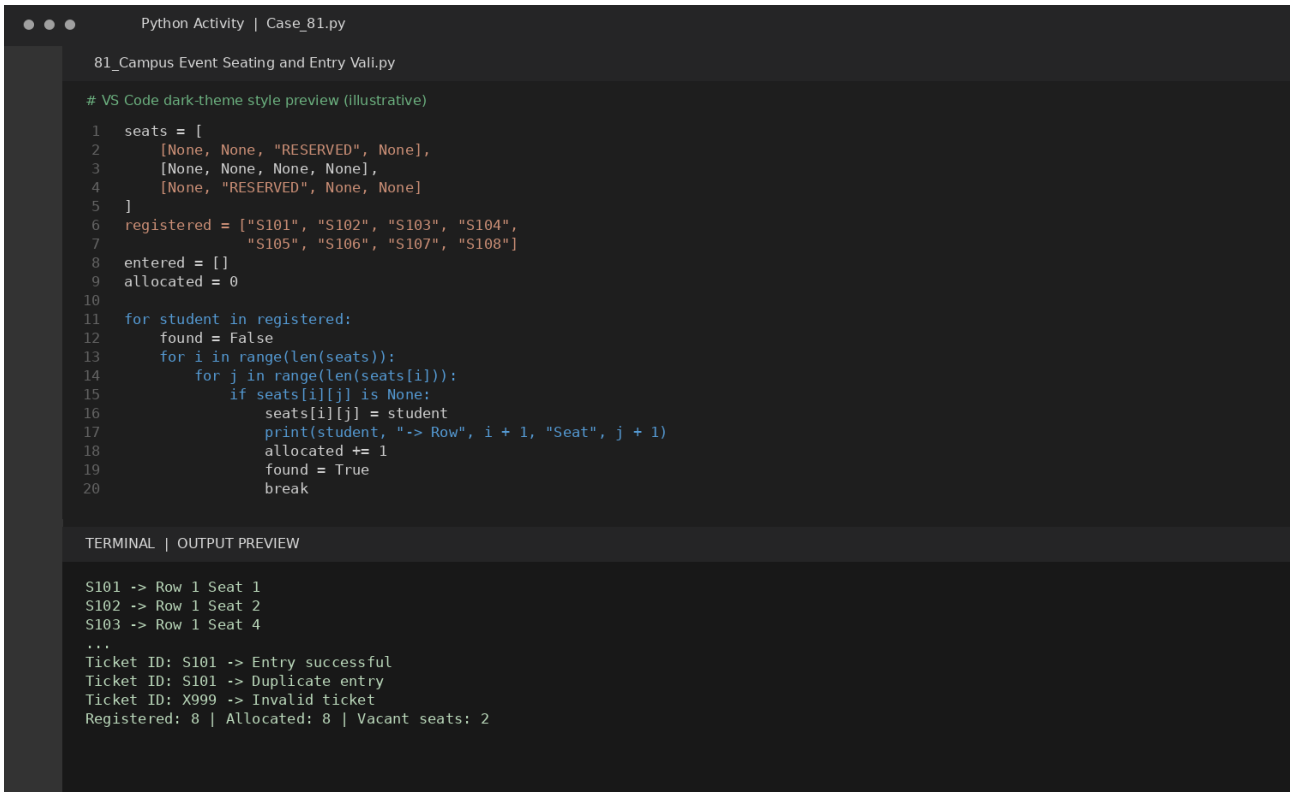
vacant = 0
for row in seats:
    for seat in row:
        if seat is None:
            vacant += 1

print("Registered:", len(registered))
print("Allocated:", allocated)
print("Entered:", len(entered))
print("Vacant seats:", vacant)
```

8. Test Cases

Test	Input / Condition	Expected Result
Registered unused ticket	S101	Entry successful
Duplicate ticket	S101 entered twice	Second attempt rejected
Unknown ticket	X999	Invalid ticket

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
S101 -> Row 1 Seat 1
S102 -> Row 1 Seat 2
S103 -> Row 1 Seat 4
...
Ticket ID: S101 -> Entry successful
Ticket ID: S101 -> Duplicate entry
Ticket ID: X999 -> Invalid ticket
Registered: 8 | Allocated: 8 | Vacant seats: 2
```

10. Conclusion

The program for Case Study 81 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.

CASE STUDY 24 – Courier Charge Optimization

1. Problem Statement

Calculate delivery charges based on parcel weight, distance and urgency, compare eligible courier services, and select the lowest-cost eligible option.

2. Objectives

• Accept parcel weight, delivery distance and delivery type.
• Calculate charges using service-specific rates.
• Enforce each service's maximum parcel weight.
• Identify the cheapest eligible service or report no eligible service.

3. Assumptions

• All service rates and maximum weights are illustrative.
• Charge = base fee + weight charge + distance charge + urgency fee.
• Normal delivery has no urgency fee; express delivery adds Rs.100.
• Weight and distance must be positive.

4. Algorithm

1. Start and read parcel weight and distance.
2. Validate positive values.
3. Read delivery type and determine urgency fee.
4. For each courier, check maximum weight.
5. Calculate and display charges for eligible services.
6. Find the lowest eligible charge and display the best service.

5. Flowchart



6. Pseudocode

```
START\nINPUT weight and distance\nIF either is not positive: DISPLAY error\nINPUT delivery type\nSET urgency fee based on delivery t
```

7. Python Source Code

```
weight = float(input("Parcel weight (kg): "))
distance = float(input("Distance (km): "))
```

```
if weight <= 0 or distance <= 0:
    print("Invalid parcel details")
else:
    print("1. Normal  2. Express")
    choice = int(input("Delivery type: "))

    if choice == 1:
        extra = 0
    elif choice == 2:
        extra = 100
    else:
        extra = -1

    if extra == -1:
        print("Invalid delivery type")
    else:
        services = ["Courier A", "Courier B", "Courier C"]
        base = [40, 50, 35]
        weight_rate = [20, 15, 25]
        distance_rate = [2, 3, 1.5]
        max_weight = [10, 20, 5]
        charges = []

        for i in range(len(services)):
            if weight <= max_weight[i]:
                charge = (base[i] + weight * weight_rate[i]
                          + distance * distance_rate[i] + extra)
                charges.append(charge)
                print(services[i], "Rs.", round(charge, 2))
            else:
                charges.append(-1)
                print(services[i], "Not eligible")

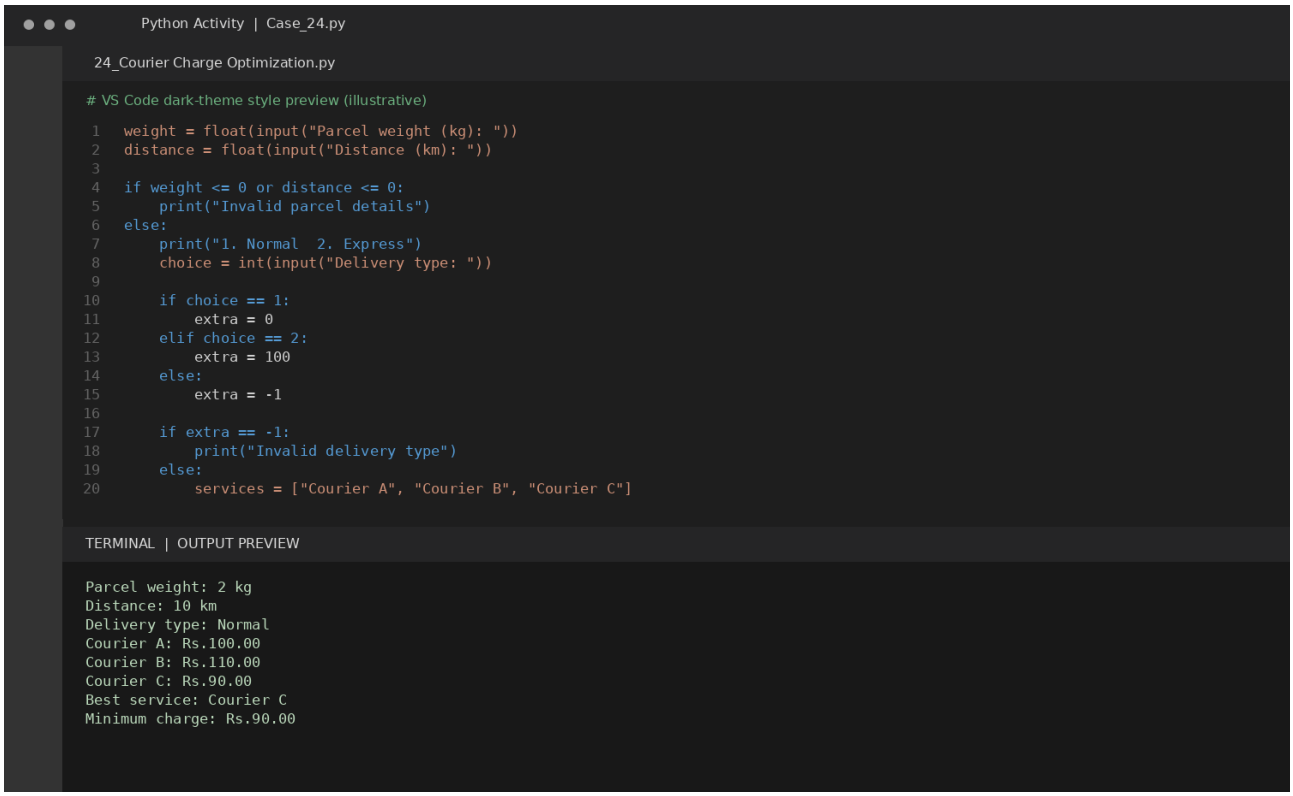
        minimum = -1
        best = ""
        for i in range(len(charges)):
            if charges[i] != -1:
                if minimum == -1 or charges[i] < minimum:
                    minimum = charges[i]
                    best = services[i]

        if minimum == -1:
            print("No eligible courier service")
        else:
            print("Best service:", best)
            print("Minimum charge: Rs.", round(minimum, 2))
```

8. Test Cases

Test	Input / Condition	Expected Result
Eligible normal parcel	2 kg; 10 km; normal	Courier C Rs.90; Courier C cheapest
Heavy parcel	12 kg; 10 km; normal	A and C ineligible; B eligible
Invalid weight	0 kg	Invalid parcel details

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
Parcel weight: 2 kg
Distance: 10 km
Delivery type: Normal
Courier A: Rs.100.00
Courier B: Rs.110.00
Courier C: Rs.90.00
Best service: Courier C
Minimum charge: Rs.90.00
```

10. Conclusion

The program for Case Study 24 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.

CASE STUDY 48 – Tuple-Based Employee Shift Conflict Detection

1. Problem Statement

Represent employee work shifts as tuples, compare time intervals, detect overlapping assignments, and report the total number of conflicts.

2. Objectives

• Store employee ID, start time and end time in tuples.
• Display shift details.
• Compare pairs of shifts for overlap.
• Count and report conflicts.

3. Assumptions

• Times are represented as integer hours on the same day.
• A shift is a half-open interval [start, end); shifts touching at an endpoint do not overlap.
• Each shift's start time is earlier than its end time.
• The sample data is illustrative.

4. Algorithm

1. Start and define a list of shift tuples.
2. Display all employee shifts.
3. Use nested loops to compare each pair once.
4. Two shifts overlap when start1 < end2 and start2 < end1.
5. Display each conflicting pair and increment conflict count.
6. Display total conflicts.

5. Flowchart



6. Pseudocode

```
START\nSET list of (employee, start, end) tuples\nDISPLAY each shift\nSET conflicts = 0\nFOR i from 0 to number of shifts - 1:\n
```

7. Python Source Code

```
shifts = [
    ("E101", 9, 13),
    ("E102", 10, 14),
    ("E103", 14, 18),
```

```
    ("E104", 11, 15),
    ("E105", 16, 20)
]

print("EMPLOYEE SHIFT DETAILS")
for employee, start, end in shifts:
    print(employee, "Start:", start, "End:", end)

print("\\nSHIFT CONFLICTS")
conflicts = 0

for i in range(len(shifts)):
    for j in range(i + 1, len(shifts)):
        emp1, start1, end1 = shifts[i]
        emp2, start2, end2 = shifts[j]

        if start1 < end2 and start2 < end1:
            print(emp1, "and", emp2, "overlap")
            conflicts += 1

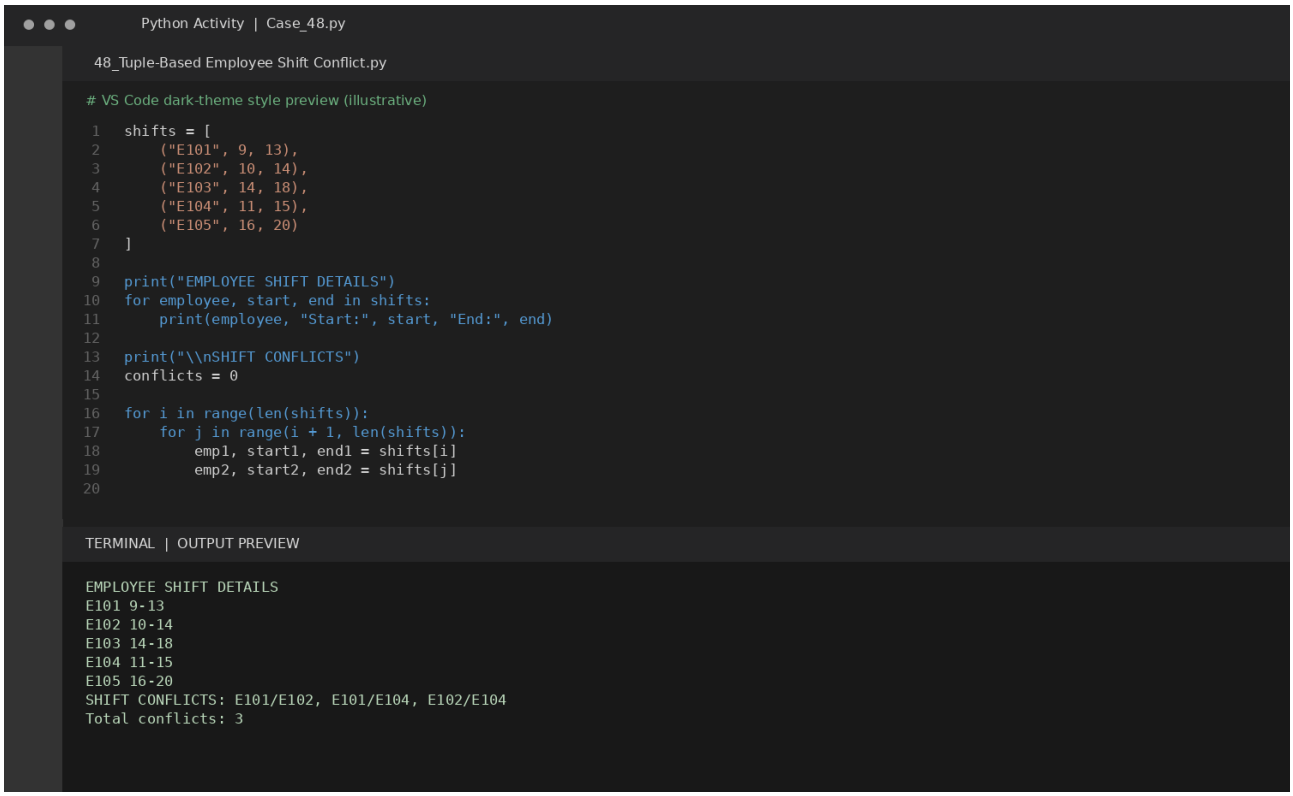
if conflicts == 0:
    print("No shift conflicts found")

print("Total conflicts:", conflicts)
```

8. Test Cases

Test	Input / Condition	Expected Result
Overlapping shifts	E101 09-13; E102 10-14	Conflict detected
Touching shifts	E101 09-13; E103 13-17	No overlap
Non-overlapping shifts	E103 14-18; E105 18-20	No overlap

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
EMPLOYEE SHIFT DETAILS
E101 9-13
E102 10-14
E103 14-18
E104 11-15
E105 16-20
SHIFT CONFLICTS: E101/E102, E101/E104, E102/E104
Total conflicts: 3
```

10. Conclusion

The program for Case Study 48 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.

CASE STUDY 94 – Disaster Relief Supply Allocation

1. Problem Statement

Compare area-wise relief-kit demand with limited available stock, prioritize urgent areas, allocate supplies according to a priority policy, and calculate unmet demand.

2. Objectives

• Store area names, demand and priority values in lists.
• Accept the available relief-kit quantity.
• Allocate supplies in priority order.
• Calculate unmet demand for each area and total remaining stock.

3. Assumptions

• Priority 1 is highest, followed by 2 and 3.
• Areas with the same priority are processed in their listed order.
• Supplies are allocated to an area up to its demand before moving to the next area.
• Each relief kit is treated as one indivisible unit.

4. Algorithm

1. Start and define areas, demand and priority lists.
2. Read available stock and validate it is non-negative.
3. For priority levels 1 to 3, visit each area.
4. Allocate the smaller of remaining stock and area demand.
5. Update stock and calculate unmet demand.
6. Display area-wise allocation and overall totals.

5. Flowchart



6. Pseudocode

```
START\nSET areas, demand and priority lists\nINPUT available stock\nIF stock < 0: DISPLAY invalid input\nFOR priority from 1 to 3:\n
```

7. Python Source Code

```
areas = ["Area A", "Area B", "Area C", "Area D"]
demand = [100, 150, 80, 120]
priority = [2, 1, 3, 1]

available_stock = int(input("Available relief kits: "))
if available_stock < 0:
```

```
    print("Invalid stock quantity")
else:
    allocated = [0, 0, 0, 0]
    unmet = [0, 0, 0, 0]

    for p in range(1, 4):
        for i in range(len(areas)):
            if priority[i] == p:
                if available_stock >= demand[i]:
                    allocated[i] = demand[i]
                else:
                    allocated[i] = available_stock

                available_stock -= allocated[i]
                unmet[i] = demand[i] - allocated[i]

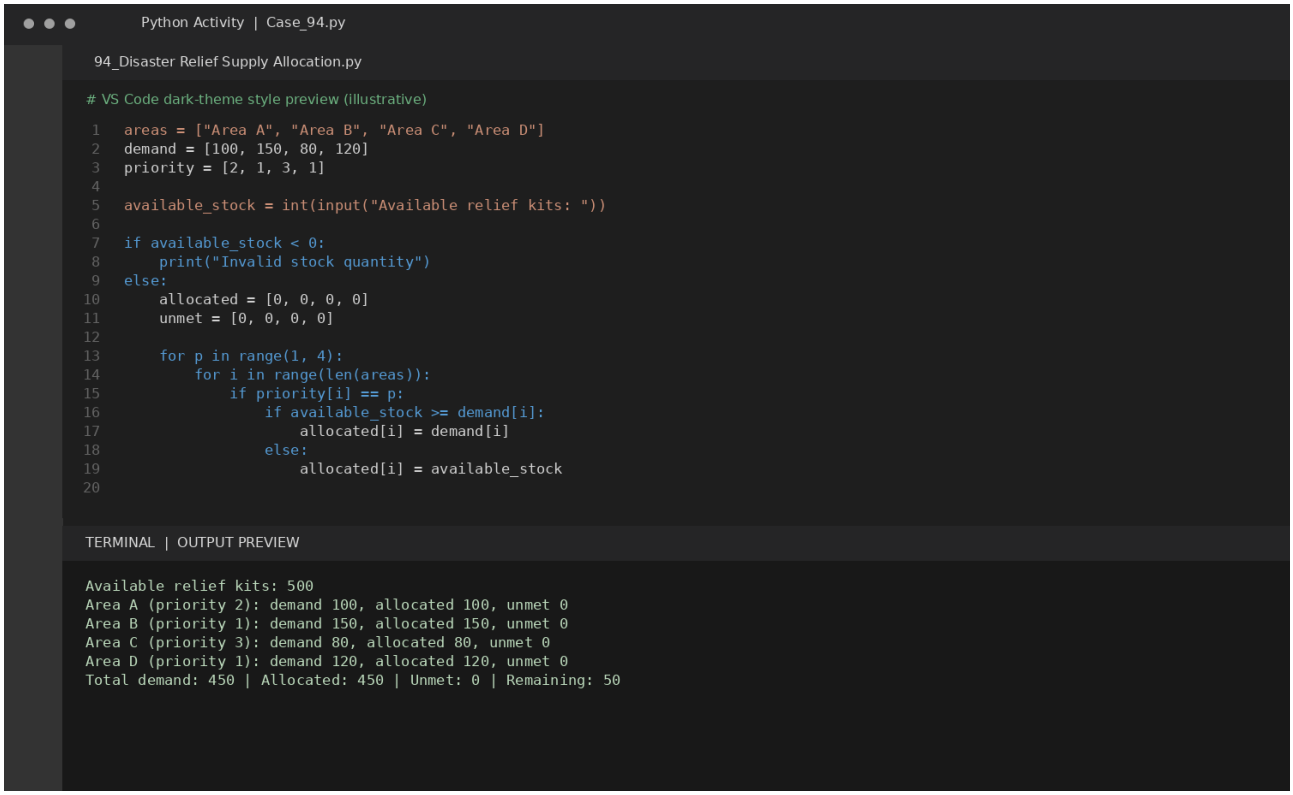
    print("DISASTER RELIEF ALLOCATION")
    for i in range(len(areas)):
        print(areas[i], "Priority:", priority[i])
        print("Demand:", demand[i])
        print("Allocated:", allocated[i])
        print("Unmet:", unmet[i])

    print("Total demand:", sum(demand))
    print("Total allocated:", sum(allocated))
    print("Total unmet:", sum(unmet))
    print("Remaining stock:", available_stock)
```

8. Test Cases

Test	Input / Condition	Expected Result
Adequate stock	Available stock 500	Demand 450; all areas fully supplied; 50 remain
Limited stock	Available stock 200	Priority-1 areas allocated first; remaining demand reported
Invalid stock	Available stock -5	Invalid stock quantity

9. VS Code Dark-Theme Screenshot (Illustrative)



Sample output:

```
Available relief kits: 500
Area A (priority 2): demand 100, allocated 100, unmet 0
Area B (priority 1): demand 150, allocated 150, unmet 0
Area C (priority 3): demand 80, allocated 80, unmet 0
Area D (priority 1): demand 120, allocated 120, unmet 0
Total demand: 450 | Allocated: 450 | Unmet: 0 | Remaining: 50
```

10. Conclusion

The program for Case Study 94 demonstrates the use of Python fundamentals to solve the stated problem. It processes the sample data, applies the stated rules, and displays a summary that can be checked against the test cases.